

# Binary Search Solution

This is a classic **sorting + binary search** problem. The key is that we don't need to check every spell-potion pair.

## C++ Code

```
class Solution {
public:
    vector<int> successfulPairs(vector<int>& spells,
                              vector<int>& potions,
                              long long success) {

        // Sort potions
        sort(potions.begin(), potions.end());

        int m = potions.size();
        vector<int> ans;

        for (long long spell : spells) {

            // We need:
            // spell * potion >= success
            //
            // Therefore:
            // potion >= success / spell

            long long required =
                (success + spell - 1) / spell;

            // Find first potion >= required
            int index = lower_bound(
                potions.begin(),
                potions.end(),
                required
            ) - potions.begin();

            // All potions from index to m-1 are successful
            ans.push_back(m - index);
        }

        return ans;
    }
};
```

## How does it work?

Suppose:

```
spells  = [5, 1, 3]
potions = [1, 2, 3, 4, 5]
success = 7
```

First, sort `potions`:

```
[1, 2, 3, 4, 5]
```

For every spell, we want:

```
spell * potion >= success
```

We can rearrange this:

```
potion >= success / spell
```

But because we're dealing with integers, we need **ceiling division**.

## Spell = 5

We need:

```
5 * potion >= 7
```

Therefore:

```
potion >= 7 / 5
```

Mathematically:

```
potion >= 1.4
```

Since potion strengths are integers:

```
potion >= 2
```

We calculate this using:

```
long long required = (success + spell - 1) / spell;
```

So:

```
required = (7 + 5 - 1) / 5
          = 11 / 5
          = 2
```

Now we have:

```
potions = [1, 2, 3, 4, 5]
           ^
           |
        first valid
```

We use:

```
lower_bound(potions.begin(), potions.end(), 2);
```

`lower_bound` gives us the index of the **first element** `>= 2`.

```
index = 1
```

There are:

```
5 - 1 = 4
```

successful potions.

So:

```
ans = [4]
```

---

## Spell = 1

We need:

```
1 * potion >= 7
```

Therefore:

```
potion >= 7
```

But:

```
[1, 2, 3, 4, 5]
```

doesn't contain anything `>= 7`.

`lower_bound(7)` returns:

```
index = 5
```

So:

```
m - index
= 5 - 5
= 0
```

Therefore:

```
ans = [4, 0]
```

---

**Spell = 3**

We need:

```
3 * potion >= 7
```

Therefore:

```
potion >= 7 / 3
```

Ceiling:

```
potion >= 3
```

So:

```
[1, 2, 3, 4, 5]
      ^
      |
    first valid
```

lower\_bound(3) gives:

```
index = 2
```

Number of successful potions:

```
5 - 2 = 3
```

Final answer:

```
[4, 0, 3]
```

---

## Why lower\_bound?

Because after sorting:

```
[1, 2, 3, 4, 5]
```

and we know we need:

```
potion >= required
```

Once we find the **first valid potion**, every potion after it is also valid.

For example:

```
required = 3

[1, 2, | 3, 4, 5]
      ^
    first valid

<----- 3 successful potions ----->
```

So instead of checking:

```
3 * 1
3 * 2
3 * 3
3 * 4
3 * 5
```

we simply find the boundary.

---

## Important: Why (success + spell - 1) / spell?

This calculates:

```
ceil(success / spell)
```

using integer arithmetic.

For example:

```
success = 7
spell = 3
```

Normal integer division:

```
7 / 3 = 2
```

But we actually need:

```
potion >= 2.333...
```

Therefore the minimum integer potion is:

```
3
```

Using:

```
(7 + 3 - 1) / 3
= 9 / 3
= 3
```

Another example:

```
success = 16
spell = 2

(16 + 2 - 1) / 2
= 17 / 2
= 8
```

Correct, because:

```
2 * 8 = 16
```

---

## Why long long?

This is important.

If:

```
spell = 100000
potion = 100000
```

then:

```
spell * potion = 10,000,000,000
```

which is larger than the range of a normal 32-bit `int`.

So we use:

```
long long
```

for `success`, `spell`, and calculations involving them.

---

## Complexity

Let:

```
n = number of spells
m = number of potions
```

Sorting potions:

```
O(m log m)
```

For each spell, binary search:

```
O(log m)
```

For `n` spells:

```
O(n log m)
```

Total:

```
O(m log m + n log m)
```

Space:

```
O(1)
```

apart from the answer array and the sorting implementation.

### The core idea to remember

Whenever you see:

```
a * b >= target
```

and one side can be sorted, think:

```
b >= ceil(target / a)
```

Then:

sort → calculate required value → lower\_bound → count elements after it.